Challenge Parkinson — prédiction du score moteur OFF

Dépôt GitHub

Luc POLO (identifiant : `@d_LO`)

Explication de la démarche

I) Premier benchmark

Pour ce premier benchmark, j'ai appliqué un **encodage one-hot** aux variables catégorielles et une **imputation par la médiane** avec indicateurs de valeurs manquantes. La validation croisée regroupe les visites par patient pour éviter toute fuite de données, et `patient_id` est exclu des features. Le meilleur modèle est un **gradient boosting** :

Résultats du premier benchmark :

LR : MSE 72,681 ± 1,229
XGBoost : MSE 56,585 ± 0,748
SVM : MSE 59,266 ± 1,040
HistGB : **MSE 55,039 ± 0,728**
RandomForest : MSE 58,789 ± 0,988

Preprocessing

Dans cette partie, je ne cite que les résultats de HistGB, car il est resté le meilleur parmi les modèles testés au fil des expérimentations.

II) Imputation des données manquantes

J'ai travaillé principalement avec des modèles d'**imputation multivariée** par régression avec gradient boosting pour prédire les valeurs manquantes de quatre variables :

- `age_at_diagnosis`
- `off`
- `on`
- `ledd`

Les autres variables numériques sont imputées par la **médiane**, tandis que les valeurs manquantes de `gene` sont représentées par la catégorie one-hot `gene_nan`. Pour chaque variable imputée, j'ai également conservé la **colonne indicatrice de valeur manquante**.

Pour entraîner les imputeurs multivariés, j'ai construit des **variables exploitant les données des visites antérieures et ultérieures** disponibles du patient (ces variables n'ont été utilisées que comme variables auxiliaires pour les imputeurs par régression, elles ont ensuite été supprimées). Exemples : `on/off_median_patient`, `time_since_first_visit`, `on/off_slope_patient`, `ledd_median_patient`, etc. Le gain est conséquent (réduction de la MSE de 17,72 %).

Récapitulatif des performances :

Méthode	MSE	std	ΔMSE (%)
Baseline	55,04	0,728	0,00 %
Imputeur multivarié	45,29	0,909	-17,72 %

Améliorations potentielles :

- Entraîner les modèles d'imputation multivariée successivement, en utilisant les prédictions des précédents comme entrées.
- Imputer les valeurs de `time_since_intake_on/off`.
- Ajouter des variables plus pertinentes pour les modèles d'imputation multivariée par régression.

III) Feature engineering

J'ai créé différentes features pour améliorer les performances du modèle. La principale valeur ajoutée des nouvelles variables créées ici est d'exploiter les informations de l'ensemble des visites d'un patient et de les partager avec chacune de ses visites. Voici l'ensemble des features qui ont été testées et retenues :

`time_since_first_visit` : temps écoulé depuis la première visite du patient.
`n_past_visits`, `n_future_visits` : nombres de visites strictement antérieures et ultérieures à la visite actuelle.
`on_slope_patient`, `off_slope_patient`, `ledd_slope_patient` : pentes annuelles des droites ajustées aux valeurs du patient en fonction de son âge.
`off_on_median_patient` : médiane de l'écart `off-on` du patient.
`on_trend_at_visit`, `off_trend_at_visit` : valeurs estimées à l'âge de la visite actuelle par les droites de tendance du patient, `off_trend_other_visits` : tendance estimée à partir des OFF observés aux autres visites.
`on_previous`, `on_next`, `off_previous`, `off_next` : valeurs ON/OFF des visites précédente et suivante disponibles.
`on_previous_gap`, `on_next_gap`, `off_previous_gap`, `off_next_gap` : écarts de temps entre la visite actuelle et les visites correspondant à ces valeurs voisines.
`n_total_visits` : nombre total de visites du patient, `off_observed_fraction_patient` : proportion de visites avec un OFF réellement observé.
`on_std_patient`, `off_std_patient` : écarts-types des valeurs ON et OFF du patient.

Éléments testés mais non concluants :

- `ledd_time_ratio_on`, `ledd_time_ratio_off` : rapports `ledd/(1+time_since_intake_on)` et `ledd/(1+time_since_intake_off)`.
- `ledd_x_time_on`, `ledd_x_time_off` : produits de la LEDD par le délai depuis la prise du traitement pour les mesures ON et OFF.
- `disease_duration` : durée de la maladie, calculée par `age-age_at_diagnosis`.
- `on_median_patient`, `off_median_patient`, `ledd_median_patient` : médianes des valeurs du patient sur ses différentes visites.
- **Effet médicamenteux :**

J'ai fait un premier essai en définissant une **courbe triangulaire commune** $h_{\theta,a}(t)$ pour approximer l'effet du traitement en fonction du temps écoulé depuis la dernière prise :

$$h_{\theta,a}(t) = a \max\left(0, \min\left(\frac{t}{t_{\text{pic}}}, \frac{t_{\text{fin}} - t}{t_{\text{fin}} - t_{\text{pic}}}\right)\right),$$
$$\theta = (t_{\text{pic}}, t_{\text{fin}}), \quad 0 < t_{\text{pic}} < t_{\text{fin}}, \ a \geq 0.$$

Les paramètres t_{pic} et t_{fin} sont sélectionnés par **grid search**, puis l'amplitude optimale a est obtenue par la **solution analytique** minimisant la somme des erreurs au carré ci-dessous.

$$\min_{a \geq 0} \sum_{i,j} \Big[(OFF - ON)_{ij} - \big(h_{\theta,a}(t_{ON,ij}) - h_{\theta,a}(t_{OFF,ij})\big) \Big]^2.$$

Cette première approche n'améliorant pas les résultats, j'ai ensuite affiné le modèle en entraînant le coefficient a individuellement pour chaque patient, noté A_i. La courbe devient :

$$h_{\theta,A_i}(t) = A_i \max\left(0, \min\left(\frac{t}{t_{\text{pic}}}, \frac{t_{\text{fin}} - t}{t_{\text{fin}} - t_{\text{pic}}}\right)\right), \quad A_i \geq 0.$$

Comme précédemment, les paramètres temporels θ sont optimisés par **grid search**, tandis que chaque amplitude A_i est estimée sur les visites éligibles du patient (souvent une ou deux seulement).

Pour les deux méthodes ci-dessus, je dispose de peu de points d'entraînement (20,1 % de toutes les visites).

Pour l'inférence de $h(t)$, 40 % des patients nécessitent l'usage d'une amplitude commune de remplacement. Les features ajoutées avec un indicateur de remplacement n'améliorent pas HistGB : MSE **14,7793** → **14,7848**.

IV) Feature selection

J'ai essayé plusieurs techniques de sélection de variables, mais aucune ne s'est avérée concluante pour mon modèle HistGB optimisé. Voici un résumé sous forme de tableau :

Méthode	k	MSE	std	ΔMSE (%)
Baseline	54	11,71	0,346	0,00 %
PCA	42	13,39	0,403	+14,34 %
SelectKBest (MI)	52	11,68	0,335	-0,27 %
SelectKBest (f_regression)	52	11,63	0,368	-0,71 %
Sélection de variables par importance (ExtraTree)	40	11,65	0,336	-0,54 %
Sélection de features récursive (CatBoost)	46	11,92	0,337	+1,79 %

V) Modeling

Je conserve HistGB, qui obtient les meilleures performances.

Une bonne inspiration ici est de combiner un **transformer** (pour modéliser les dépendances entre les visites) et le HistGB que j'utilise maintenant.

Je crée donc un encodeur à l'architecture simple. Je fais le choix d'un **encodeur bidirectionnel**, car on peut, dans le train comme dans le test set, accéder aux visites futures comme passées des patients. Cet encodeur prend en entrée les séquences des visites des patients en appliquant du padding pour les séquences trop courtes et auxquelles on a ajouté dans les features le temps depuis la première visite pour encoder la dimension temporelle.

L'encodeur est entraîné à prédire les scores OFF de chaque visite. J'utilise non pas ses prédictions, mais les **embeddings** extraits du dernier bloc d'attention dans HistGB. Ces embeddings sont ajoutés aux features précédemment construites pour HistGB, qui réalise la prédiction finale.

Les améliorations notables au cours du modeling :

- Optimisation des paramètres de l'encodeur (dont les paramètres d'architecture, comme le nombre de layers et de têtes d'attention) par **optimisation bayésienne** avec cross validation pour estimer la performance de chaque combinaison de paramètres + hold-out set pour choisir, pour chaque test, la meilleure valeur du nombre d'époques. −10 % de MSE.
- Utiliser un scheduler : **Cosine annealing with warm restarts** (après ré-optimisation des paramètres avec Optuna). −8,25 % de MSE. Voir le schéma de cette architecture testée en annexe.
- Utilisation de l'encodeur seul une fois son entraînement et ses paramètres optimisés. −1,39 % de MSE.

Récapitulatif des performances :

Modèle / étape	MSE	std	ΔMSE (%)
HistGB (baseline)	11,73	0,308	0,00 %
Encodeur + HistGB	8,33	0,319	-29,02 %
Optimisation des paramètres (Optuna)	7,50	0,412	-36,11 %
Ajout de cosine annealing avec warm restart + Optuna	6,88	0,283	-41,38 %
Encodeur seul	6,78	0,225	-42,20 %

Éléments testés mais non concluants :

- Utiliser un token CLS (pas très utile, mais curiosité). +1 % de MSE.
- Utiliser l'ensemble ou une partie des features créées durant le features engineering à l'entrée de l'encodeur. +1,8 % de MSE.
- Ajout d'un encodage sinusoïdal 64D du temps depuis la première visite aux features projetées, au lieu de l'ajout du temps écoulé depuis la première visite dans les features. MSE quasiment inchangée.
- Utiliser les prédictions de score OFF prédites par l'encodeur en plus des embeddings dans les features de HistGB. Cependant, cela nécessite de diviser le trainset en deux pour ne pas entraîner les modèles sur le même dataset. +5 % de MSE.
- Régression Ridge à la place de HistGB. MSE quasiment inchangée.

Améliorations potentielles :

- Faire des essais plus fins sur les variables issues du feature engineering à mettre en entrée de l'encodeur.

VI) Post-Processing

J'ai testé deux méthodes pour rendre les trajectoires prédites non décroissantes : la **régression isotone** et le **lissage monotone**. Leur impact sur la MSE de l'encodeur seul est négligeable, mais je conserve ce post-traitement par cohérence.

Ces deux méthodes avaient significativement plus d'impact lorsqu'elles étaient appliquées au modèle encodeur + HistGB (−0,96 % de MSE pour la régression isotone et −1,7 % pour le lissage monotone).

Récapitulatif des performances :

Méthode	MSE calc.	std	ΔMSE (%)
Baseline	6,78	0,225	0,00 %
Régression isotone	6,78	0,223	-0,05 %
Lissage monotone	6,76	0,218	-0,36 %

VII) Résultats

Étape	MSE	std	ΔMSE (%)
I) Premier benchmark	55,04	0,728	—
II) Imputation multivariée	45,29	0,909	-17,72 %
III) Feature engineering	11,71	0,346	-74,14 %
IV) Feature selection	11,63	0,368	-0,71 %
V) Modeling	6,78	0,225	-41,67 %
VI) Post-processing	6,76	0,218	-0,36 %

L'écart avec la MSE de la soumission finale du challenge s'explique notamment par la différence de protocole d'entraînement : les scores de ce rapport proviennent de modèles entraînés sur 80 % des données d'entraînement, tandis que le modèle de la soumission finale a été entraîné sur 100 % de ces données.

Annexe

Architecture Encodeur + HistGB testée

